

Week 04: Fuzzy Logic and Fuzzy Control

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-04/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-04/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-04/lab.html>

Colab notebook with Python step 4 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-04/NB04_fuzzy_control.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Operators describe processes in words: The level is a bit low and falling slowly, so open the valve a little. Fuzzy logic turns such statements into computation. Zadeh introduced fuzzy sets, whose members belong to a degree between zero and one [1], and linguistic variables that take words as values [2]. Mamdani and Assilian used them to control a steam engine with rules taken from operators [3], and within a decade fuzzy controllers ran cement kilns and a subway line [4, 5]. This week builds a fuzzy inference system from scratch, uses it to control the level of a tank, compares Mamdani and Takagi-Sugeno models, and shows how adaptive neuro-fuzzy systems learn their parameters from data [6, 7].

Estimated study time: 9 to 11 hours.

Learning outcomes

By the end of the week, students are expected to define fuzzy sets with triangular, trapezoidal and Gaussian membership functions, to apply fuzzy complement, intersection and union, to carry out Mamdani inference by hand from fuzzification to centroid defuzzification, to design a rule table for a two-input controller and interpret its control surface, to distinguish Mamdani from Takagi-Sugeno models, and to implement and simulate a fuzzy controller in Python, both from scratch and with scikit-fuzzy.

Python in this week

These notes close with Python step 4: NumPy arrays and first plots. It covers NumPy arrays, element-wise arithmetic, functions on arrays, boolean masks and Matplotlib plots, applied to membership functions and defuzzification [8, 9]. In the course materials, the step is taught at the start of the Colab

notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Vagueness and fuzzy sets

Week 3 ended with a weakness of crisp rules: A threshold of 70 degrees makes a reading of 69.9 normal and a reading of 70.1 overheated. Engineers rarely think in such steps. A temperature is more or less high, a load is fairly heavy, a surface is quite rough. Zadeh proposed fuzzy sets to represent such graded concepts [1]. A fuzzy set A on a universe of discourse X is defined by a membership function that assigns every element x a degree between 0 and 1. A room at 23 degrees may belong to the set of comfortable rooms with degree 0.6 and to the set of warm rooms with degree 0.125.

Vagueness is not probability. A membership degree of 0.6 does not say that the room is comfortable with probability 0.6. It says that 23 degrees fits the concept “comfortable” to degree 0.6, and there is nothing random about it. Probability describes uncertainty about whether an event happens; fuzzy membership describes how well a precisely known value fits an imprecise concept. The two can be combined, but they answer different questions.

Common membership functions are triangular, defined by a left foot, a peak and a right foot; trapezoidal, with a flat top; and Gaussian, with a centre and a width. The support of a fuzzy set is where its membership is positive, the core is where it equals one, and an alpha-cut is the crisp set of elements whose membership is at least alpha. A linguistic variable, such as tank level, takes linguistic values, such as low, okay and high, each represented by a fuzzy set on the same universe [2].

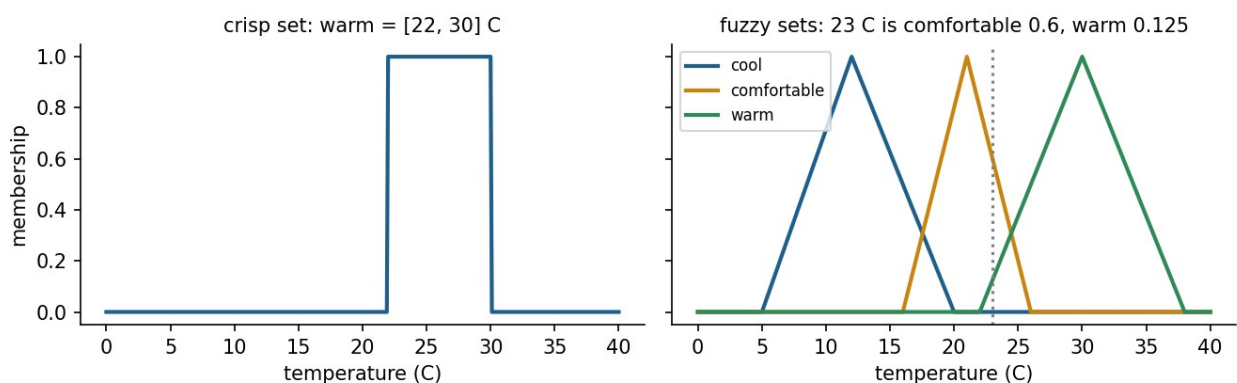


Figure 4.1. A crisp set and fuzzy sets for the concept “warm room”. The fuzzy sets overlap, so a temperature can belong partly to two neighbouring concepts [1].

Check your understanding. A water temperature of 58 degrees has membership 0.7 in the fuzzy set “hot”. What does this mean?

- A. The water is hot with probability 0.7
- B. 58 degrees fits the concept “hot” to degree 0.7
- C. 70 percent of the water is hot
- D. The measurement has an error of 0.7 degrees

Answer: B. Membership measures compatibility with a vague concept, not the probability of a random event.

Formulas: Membership functions

A fuzzy set A on a universe of discourse X , for example the room temperatures from 0 to 40 degrees, is defined by a membership function

$$\mu_A: X \rightarrow [0, 1]$$

that gives every element x a degree of membership $\mu_A(x)$ between 0 and 1 [1]. The Greek letter mu, μ , is the usual symbol. A crisp set is the special case in which $\mu_A(x)$ is either 0 or 1. Three shapes are common. A triangular set with left foot a , peak b and right foot c rises linearly from a to b and falls linearly from b to c :

$$\mu(x; a, b, c) = \max\left(\min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right), 0\right)$$

The first fraction is the rising side, the second the falling side, the minimum picks the side on which x lies, and the maximum with 0 sets the membership to zero outside the feet. When $a=b$, the rising side is replaced by 1 and the set becomes a left shoulder, which keeps full membership at the low end of the universe; $b=c$ gives a right shoulder. A trapezoidal set has a flat top from b to c :

$$\mu(x; a, b, c, d) = \max\left(\min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right), 0\right)$$

A Gaussian set with centre m and width σ (sigma) is smooth everywhere:

$$\mu(x; m, \sigma) = e^{-(x-m)^2/(2\sigma^2)}$$

Three crisp sets describe a fuzzy set. The support holds the elements with positive membership, the core those with full membership, and the alpha-cut A_α those with membership of at least α :

$$\text{supp}(A) = \{x : \mu_A(x) > 0\}, \text{core}(A) = \{x : \mu_A(x) = 1\}, A_\alpha = \{x : \mu_A(x) \geq \alpha\}$$

Worked example: How warm is a room at 23 degrees?

Figure 4.1 and the notebook use three triangular sets on the room temperature: cool (5, 12, 20), comfortable (16, 21, 26) and warm (22, 30, 38), each written as (left foot, peak, right foot).

Step 1. Cool. The value 23 lies above the right foot 20, so the falling side is negative,

$$\frac{20-23}{20-12} = \frac{-3}{8} = -0.375, \text{ and the maximum with 0 gives } \mu_{\text{cool}}(23) = 0.$$

Step 2. Comfortable. Both sides are evaluated and the smaller one is kept:

$$\frac{23-16}{21-16} = \frac{7}{5} = 1.4, \frac{26-23}{26-21} = \frac{3}{5} = 0.6, \mu_{\text{comfortable}}(23) = \min(1.4, 0.6) = 0.6$$

Step 3. Warm.

$$\frac{23-22}{30-22} = \frac{1}{8} = 0.125, \frac{38-23}{38-30} = \frac{15}{8} = 1.875, \mu_{\text{warm}}(23) = \min(0.125, 1.875) = 0.125$$

The room is comfortable to degree 0.6 and warm to degree 0.125, while the crisp set “warm” of Figure 4.1, the interval from 22 to 30 degrees, would give it full membership.

Step 4. Other shapes. A Gaussian set “comfortable” with centre 21 and width 2.5, as in the notebook, gives

$$\frac{23-21}{2.5} = 0.8, 0.8^2 = 0.64, \mu = e^{-0.64/2} = e^{-0.32} = 0.7261$$

and a trapezoidal set with feet 16 and 26 and a flat top from 20 to 22 gives

$$\mu = \max\left(\min\left(\frac{23-16}{20-16}, 1, \frac{26-23}{26-22}\right), 0\right) = \max(\min(1.75, 1, 0.75), 0) = 0.75$$

Step 5. Support, core and alpha-cut of the triangular set “comfortable”. The support is the open interval from 16 to 26, and the core is the single temperature 21. For $\alpha = 0.5$, the rising side reaches 0.5 where $\frac{x-16}{5} = 0.5$, that is at $x = 16 + 0.5 \times 5 = 18.5$, and the falling side where $\frac{26-x}{5} = 0.5$, at $x = 26 - 0.5 \times 5 = 23.5$. The alpha-cut is the interval from 18.5 to 23.5 degrees.

Operations on fuzzy sets

The classical set operations generalise. The complement NOT A has membership 1 minus the membership of A. The intersection A AND B is usually computed with the minimum of the two memberships, and the union A OR B with the maximum [1]. Other choices are possible: The product is also a valid intersection, called a t-norm, and the probabilistic sum $a + b - ab$ is a valid union, called an s-norm. The choice changes the smoothness of the resulting controller but not the principle. Some classical laws fail on purpose: A AND NOT A need not be empty, because a temperature can be somewhat warm and somewhat not warm at the same time.

Formulas: Complement, intersection and union

For fuzzy sets A and B on the same universe, the standard operations of Zadeh work point by point [1]:

$$\mu_{\neg A}(x) = 1 - \mu_A(x)$$

$$\mu_{A \wedge B}(x) = \min(\mu_A(x), \mu_B(x)), \mu_{A \vee B}(x) = \max(\mu_A(x), \mu_B(x))$$

For degrees of 0 and 1 they reproduce the truth tables of Week 3. The minimum is one member of a family of intersections called t-norms, functions $T(a, b)$ of two degrees that are commutative, associative, non-decreasing in each argument and satisfy $T(a, 1) = a$. The product $T(a, b) = ab$ is another t-norm. Every t-norm has a matching union, its s-norm, obtained through the complement:

$$S(a, b) = 1 - T(1 - a, 1 - b)$$

For the minimum this gives the maximum. For the product it gives the probabilistic sum,

$$S(a, b) = 1 - (1 - a)(1 - b) = a + b - ab$$

With these pairs the laws of De Morgan hold, for example $\neg(A \wedge B) = \neg A \vee \neg B$, whereas the law of contradiction does not: $\min(\mu, 1 - \mu)$ is zero only when μ is 0 or 1 [1, 12].

Worked example: Combining the two memberships of the room

At 23 degrees the room is comfortable with $a = 0.6$ and warm with $b = 0.125$.

Step 1. Complements. Not warm: $1 - 0.125 = 0.875$. Not comfortable: $1 - 0.6 = 0.4$.

Step 2. Comfortable and warm. With the minimum, $\min(0.6, 0.125) = 0.125$. With the product, $0.6 \times 0.125 = 0.075$.

Step 3. Comfortable or warm. With the maximum, $\max(0.6, 0.125) = 0.6$. With the probabilistic sum, $0.6 + 0.125 - 0.075 = 0.65$.

Step 4. De Morgan. On the left side, $1 - \min(0.6, 0.125) = 1 - 0.125 = 0.875$. On the right side, $\max(1 - 0.6, 1 - 0.125) = \max(0.4, 0.875) = 0.875$. The two sides agree.

Step 5. A set and its complement. Comfortable and not comfortable: $\min(0.6, 0.4) = 0.4$, not 0. Comfortable or not comfortable: $\max(0.6, 0.4) = 0.6$, not 1. The room is partly comfortable and partly not, which is exactly the vagueness the fuzzy set expresses. Part A of the [interactive lab](#) evaluates triangular, trapezoidal and Gaussian sets and these operations for any input.

Mamdani inference

A fuzzy rule has linguistic conditions and a linguistic conclusion: IF level error is low AND level rate is none THEN valve change is open fast. Mamdani inference evaluates a rule base in five steps [3, 10]. First, fuzzification computes the membership of each crisp input in each of its fuzzy sets. Second, each rule receives a firing strength, the minimum of the memberships in its conditions. Third, implication clips the output fuzzy set of each rule at its firing strength. Fourth, aggregation combines the clipped sets of all rules with the maximum. Fifth, defuzzification turns the aggregated fuzzy set into one crisp output, most often with the centroid, the centre of gravity of the area under the aggregated membership function.

Animation: Inside a Mamdani controller. Move the two inputs of a tank level controller. The bars show fuzzification, the clipped shapes show the rules that fire, and the vertical line marks the centroid that becomes the valve command. Run it on the [lecture page](#).

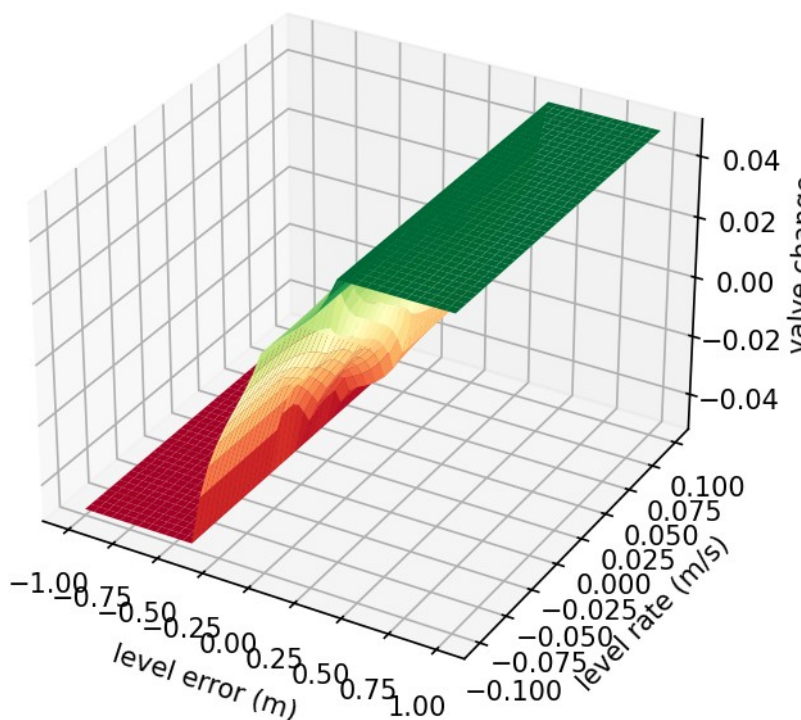


Figure 4.2. Control surface of the five-rule tank controller of the notebook. The fuzzy controller is a smooth nonlinear map from level error and level rate to valve change.

The controller defined by a rule base is a nonlinear function from inputs to output, and its control surface can be drawn for two inputs. Overlapping membership functions make the surface smooth; more sets and rules add detail. A rule table, with one input along the rows and the other along the columns, is the usual design tool: Every cell holds the conclusion for one combination of linguistic values, and gaps in the table leave parts of the input space without any action. Symmetry of the table usually leads to symmetric behaviour of the loop.

Check your understanding. Two rules fire with strengths 0.3 and 0.8 and conclude “open slow” and “open fast”. What happens in Mamdani inference before defuzzification?

- A. Only the stronger rule is kept
- B. Each output set is clipped at its rule strength and the clipped sets are combined with the maximum
- C. The strengths are added to 1.1 and the output is saturated
- D. The inputs are fuzzified again with new membership functions

Answer: B. Implication clips each consequent at its firing strength and aggregation takes the pointwise maximum of the clipped sets.

Formulas: The five steps of Mamdani inference

Consider rules with two inputs, IF x is A_k AND y is B_k THEN z is C_k , for $k = 1, \dots, K$, and crisp inputs x_0 and y_0 [3, 10].

1. Fuzzification computes the memberships $\mu_{A_k}(x_0)$ and $\mu_{B_k}(y_0)$ of the inputs in every set used by the rules.

2. Firing strength. The AND of a rule is the minimum of its memberships:

$$w_k = \min(\mu_{A_k}(x_0), \mu_{B_k}(y_0))$$

3. Implication clips the output set of each rule at its firing strength:

$$\mu'_k(z) = \min(w_k, \mu_{C_k}(z))$$

4. Aggregation joins the clipped sets of all rules with the maximum:

$$\mu(z) = \max_k \mu'_k(z)$$

5. Defuzzification by the centroid returns the centre of gravity of the area under $\mu(z)$. On a grid of output values $z_1, z_2, \dots, z_N$ the integrals become sums:

$$z^* = \frac{\int z \mu(z) dz}{\int \mu(z) dz} \approx \frac{\sum_{j=1}^N z_j \mu(z_j)}{\sum_{j=1}^N \mu(z_j)}$$

The integral sign $\int$ adds up infinitely many thin strips; the sum on the right adds up the values at the grid points, and a finer grid brings it closer to the integral.

Worked example: The tank level controller of the animation

The controller of the animation and the notebook has the level error e (set point minus level, in metres) and the level rate r (in metres per second) as inputs and the change of the valve opening per second as output. Its sets, written as (left foot, peak, right foot), are high $(-1, -1, 0)$, okay $(-0.4, 0, 0.4)$ and low $(0, 1, 1)$ for the error; negative $(-0.1, -0.1, 0)$, none $(-0.03, 0, 0.03)$ and positive $(0, 0.1, 0.1)$ for the rate; and close fast $(-0.07, -0.05, -0.03)$, close slow $(-0.035, -0.02, -0.005)$, no change $(-0.012, 0, 0.012)$, open slow $(0.005, 0.02, 0.035)$ and open fast $(0.03, 0.05, 0.07)$ for the output. The five rules are: R1 IF e is low THEN open fast; R2 IF e is high THEN close fast; R3 IF e is okay AND r is none THEN no change; R4 IF e is okay AND r is positive THEN close slow; R5 IF e is okay AND r is negative THEN open slow. The inputs are $e = 0.25$ m, the level is a quarter of a metre below the set point, and $r = 0.02$ m/s, the level is rising.

Step 1. Fuzzification of the error.

$$\mu_{\text{high}}(0.25) = \max\left(\min\left(1, \frac{0 - 0.25}{0 - (-1)}\right), 0\right) = \max(-0.25, 0) = 0$$

$$\mu_{\text{okay}}(0.25) = \max\left(\min\left(\frac{0.25 + 0.4}{0.4}, \frac{0.4 - 0.25}{0.4}\right), 0\right) = \min(1.625, 0.375) = 0.375$$

$$\mu_{\text{low}}(0.25) = \max\left(\min\left(\frac{0.25 - 0}{1 - 0}, 1\right), 0\right) = 0.25$$

Step 2. Fuzzification of the rate.

$$\mu_{\text{negative}}(0.02) = \max\left(\min\left(1, \frac{0 - 0.02}{0.1}\right), 0\right) = \max(-0.2, 0) = 0$$

$$\mu_{\text{none}}(0.02) = \max\left(\min\left(\frac{0.02 + 0.03}{0.03}, \frac{0.03 - 0.02}{0.03}\right), 0\right) = \min(1.6667, 0.3333) = 0.3333$$

$$\mu_{\text{positive}}(0.02) = \max\left(\min\left(\frac{0.02}{0.1}, 1\right), 0\right) = 0.2$$

Step 3. Firing strengths. R1: $w_1 = 0.25$. R2: $w_2 = 0$. R3: $w_3 = \min(0.375, 0.3333) = 0.3333$. R4: $w_4 = \min(0.375, 0.2) = 0.2$. R5: $w_5 = \min(0.375, 0) = 0$. Rules R2 and R5 do not fire.

Step 4. Implication on a grid. The output universe from -0.07 to 0.07 is sampled every 0.01 , which gives 15 grid points. Close slow is positive at three of them: $\frac{-0.03 + 0.035}{0.015} = 0.3333$ at -0.03 , 1 at

-0.02 and $\frac{-0.005 - (-0.01)}{0.015} = 0.3333$ at -0.01 . Clipped at $w_4 = 0.2$, all three become

$\min(0.2, 0.3333) = \min(0.2, 1) = 0.2$. No change is positive at -0.01 with $\frac{-0.01 + 0.012}{0.012} = 0.1667$, at

0 with 1 and at 0.01 with $\frac{0.012 - 0.01}{0.012} = 0.1667$; clipped at $w_3 = 0.3333$ these become 0.1667 , 0.3333 and

0.1667 . Open fast is positive at 0.04 with $\frac{0.04 - 0.03}{0.02} = 0.5$, at 0.05 with 1 and at 0.06 with

$\frac{0.07 - 0.06}{0.02} = 0.5$; clipped at $w_1 = 0.25$ all three become 0.25 .

Step 5. Aggregation and the products for the centroid. The column $\mu(z)$ holds the maximum of the three clipped sets; at $z = -0.01$, for example, $\max(0.2, 0.1667, 0) = 0.2$.

z	close slow	no change	open fast	$\mu(z)$	$z \mu(z)$
-0.07	0	0	0	0	0
-0.06	0	0	0	0	0
-0.05	0	0	0	0	0
-0.04	0	0	0	0	0
-0.03	0.2	0	0	0.2	-0.006
-0.02	0.2	0	0	0.2	-0.004
-0.01	0.2	0.1667	0	0.2	-0.002
0	0	0.3333	0	0.3333	0
0.01	0	0.1667	0	0.1667	0.001667
0.02	0	0	0	0	0
0.03	0	0	0	0	0
0.04	0	0	0.25	0.25	0.01
0.05	0	0	0.25	0.25	0.0125

z	close slow	no change	open fast	$\mu(z)$	$z \mu(z)$
0.06	0	0	0.25	0.25	0.015
0.07	0	0	0	0	0

Step 6. Centroid.

$$\sum_j \mu(z_j) = 0.2 + 0.2 + 0.2 + 0.3333 + 0.1667 + 0.25 + 0.25 + 0.25 = 1.85$$

$$\sum_j z_j \mu(z_j) = (-0.006) + (-0.004) + (-0.002) + 0.001667 + 0.01 + 0.0125 + 0.015 = 0.027167$$

$$z^* = \frac{0.027167}{1.85} = 0.014685$$

The valve opens by about 0.0147 of its range per second: The level is low, which calls for opening, and the rising level holds the action back. The grid of 15 points is coarse. A grid every 0.005, with 29 points, gives 0.0177, and the 561 points of the notebook give 0.0169, the value that the integral approaches. Part B of the [interactive lab](#) repeats all steps for any inputs and grid.

Takagi-Sugeno models and learning fuzzy systems

Takagi and Sugeno replaced the fuzzy conclusion by a function of the inputs: IF x is A AND y is B THEN $z = p x + q y + r$ [6]. The output is the average of the rule outputs weighted by their firing strengths, so no defuzzification of a shape is needed. A zero-order Sugeno model has constant conclusions and behaves much like a Mamdani model with narrow output sets; a first-order model blends local linear models, which makes it attractive for modelling and for gain scheduling of controllers.

Because the Sugeno output is a differentiable function of its parameters, the parameters can be learned from data. ANFIS represents a Sugeno system as a five-layer adaptive network and trains the premise parameters with gradient descent and the linear consequent parameters with least squares [7]. Metaheuristic training is an alternative when gradients are unreliable: In one study, ANFIS supported by the vortex optimisation algorithm forecast chaotic electroencephalogram time series [11]. Such neuro-fuzzy systems link this week to the optimisation of Weeks 5 and 6 and to the neural networks of Week 10.

Check your understanding. What distinguishes a first-order Takagi-Sugeno rule from a Mamdani rule?

- A. Its conditions are crisp
- B. Its conclusion is a linear function of the inputs instead of a fuzzy set
- C. It cannot be combined with other rules
- D. It uses the maximum instead of the minimum for AND

Answer: B. The consequent $z = p x + q y + r$ is a local linear model; the rule outputs are averaged with the firing strengths as weights.

Formulas: Takagi-Sugeno models and ANFIS

A first-order Takagi-Sugeno rule has a linear function as its conclusion [6]:

$$\text{IF } x \text{ is } A_k \text{ AND } y \text{ is } B_k \text{ THEN } z_k = p_k x + q_k y + r_k$$

A zero-order rule has a constant conclusion, $z_k = r_k$. The firing strength w_k is computed as in Mamdani inference, with the minimum or with the product of the memberships, and the output is the weighted average of the rule outputs:

$$z = \frac{\sum_{k=1}^K w_k z_k}{\sum_{k=1}^K w_k} = \sum_{k=1}^K \bar{w}_k z_k, \bar{w}_k = \frac{w_k}{\sum_{j=1}^K w_j}$$

The normalised strengths $\bar{w}_k$ add up to one. ANFIS computes the same output with a five-layer network: Layer 1 evaluates the membership functions, layer 2 multiplies them into the strengths w_k , layer 3 normalises them, layer 4 forms $\bar{w}_k z_k$, and layer 5 adds these up [7]. Its hybrid learning rule uses two facts. First, once the membership functions are fixed, z is linear in the consequent parameters, so these follow from a least-squares fit to the training pairs (x_i, t_i) . Second, the membership parameters, such as the centre c of a Gaussian set, are improved by gradient descent on the squared error $E = \frac{1}{2}(z - t)^2$ with a learning rate η (eta):

$$c \leftarrow c - \eta \frac{\partial E}{\partial c}$$

The symbol $\partial E / \partial c$ is the partial derivative, the rate at which E changes when only c changes. For two rules with Gaussian sets of width σ , the chain rule gives

$$\frac{\partial E}{\partial c_1} = (z - t) \times (z_1 - z_2) \times \bar{w}_1 \bar{w}_2 \times \frac{x - c_1}{\sigma^2}$$

because $z = \bar{w}_1 z_1 + (1 - \bar{w}_1) z_2$ changes by $z_1 - z_2$ per unit of $\bar{w}_1$, and $\bar{w}_1$ changes by $\bar{w}_1 \bar{w}_2 (x - c_1) / \sigma^2$ per unit of c_1 .

Worked example: A zero-order Sugeno version of the tank controller

Replacing each output set of the controller by its peak gives the constants close fast -0.05 , close slow -0.02 , no change 0 , open slow 0.02 and open fast 0.05 . With the firing strengths of the Mamdani example, $w = (0.25, 0, 0.3333, 0.2, 0)$:

$$z = \frac{0.25 \times 0.05 + 0 \times (-0.05) + 0.3333 \times 0 + 0.2 \times (-0.02) + 0 \times 0.02}{0.25 + 0 + 0.3333 + 0.2 + 0}$$

$$z = \frac{0.0125 + 0 + 0 - 0.004 + 0}{0.7833} = \frac{0.0085}{0.7833} = 0.010852$$

The Sugeno controller needs no output shapes and no grid, and its output, 0.010852 , is close to the Mamdani centroid of 0.0169 . The difference comes from the widths and overlaps of the Mamdani output sets.

Worked example: One learning cycle of a small ANFIS

A sensor has the input x and the measured output t at three points, $(2, 3)$, $(5, 5)$ and $(8, 8)$. A model with two zero-order rules, IF x is small THEN $z = k_1$ and IF x is large THEN $z = k_2$, uses Gaussian sets with centres $c_1 = 0$ and $c_2 = 10$ and width $\sigma = 4$, so that $2\sigma^2 = 32$.

Step 1. Layers 1 to 3 for each training point. For $x = 2$: $w_1 = e^{-(2-0)^2/32} = e^{-0.125}$ and $w_2 = e^{-(2-10)^2/32} = e^{-2}$. All three points:

x	t	w_1	w_2	$\bar{w}_1$	$\bar{w}_2$
2	3	$e^{-0.125} = 0.882497$	$e^{-2} = 0.135335$	0.867036	0.132964

x	t	w_1	w_2	$\bar{w}_1$	$\bar{w}_2$
5	5	$e^{-0.7812}=0.457856$	$e^{-0.7812}=0.457856$	0.5	0.5
8	8	$e^{-2}=0.135335$	$e^{-0.125}=0.882497$	0.132964	0.867036

For $x=2$, for example, $\bar{w}_1=0.882497/(0.882497+0.135335)=0.882497/1.017832=0.867036$.

Step 2. Least squares for the consequents. The model output is $z_i=\bar{w}_{1i}k_1+\bar{w}_{2i}k_2$. Setting the derivatives of $\sum_i (z_i-t_i)^2$ with respect to k_1 and k_2 to zero gives the normal equations

$$\left(\sum_i \bar{w}_{1i}^2\right)k_1+\left(\sum_i \bar{w}_{1i}\bar{w}_{2i}\right)k_2=\sum_i \bar{w}_{1i}t_i$$

$$\left(\sum_i \bar{w}_{1i}\bar{w}_{2i}\right)k_1+\left(\sum_i \bar{w}_{2i}^2\right)k_2=\sum_i \bar{w}_{2i}t_i$$

The sums over the three points are

$$\sum_i \bar{w}_{1i}^2=0.867036^2+0.5^2+0.132964^2=1.01943, \sum_i \bar{w}_{2i}^2=1.01943$$

$$\sum_i \bar{w}_{1i}\bar{w}_{2i}=0.867036 \times 0.132964+0.5 \times 0.5+0.132964 \times 0.867036=0.48057$$

$$\sum_i \bar{w}_{1i}t_i=0.867036 \times 3+0.5 \times 5+0.132964 \times 8=6.16482$$

$$\sum_i \bar{w}_{2i}t_i=0.132964 \times 3+0.5 \times 5+0.867036 \times 8=9.83518$$

Cramer's rule solves the two equations with the determinant

$$D=1.01943 \times 1.01943 - 0.48057^2=0.80829$$

$$k_1=\frac{6.16482 \times 1.01943 - 0.48057 \times 9.83518}{D}=\frac{1.55811}{0.80829}=1.927662$$

$$k_2=\frac{1.01943 \times 9.83518 - 0.48057 \times 6.16482}{D}=\frac{7.06365}{0.80829}=8.739005$$

The fitted outputs are 2.833325, 5.333334 and 7.833342 for the targets 3, 5 and 8.

Step 3. One gradient step for the centre c_1 at the first point. At $x=2$ the error is

$z-t=2.833325-3=-0.166675$. The factors of the chain rule are

$z_1-z_2=k_1-k_2=1.927662-8.739005=-6.811343$, $\bar{w}_1\bar{w}_2=0.867036 \times 0.132964=0.115285$ and $(x-c_1)/\sigma^2=2/16=0.125$:

$$\frac{\partial E}{\partial c_1}=-0.166675 \times (-6.811343) \times 0.115285 \times 0.125=0.01636$$

With $\eta=0.5$ the centre moves to $c_1=0-0.5 \times 0.01636=-0.00818$. The step is small because the first rule already fits this point well. ANFIS repeats such least-squares and gradient steps over all points until the error stops falling [7]. Part C of the [interactive lab](#) runs these cycles on editable data.

Fuzzy control in engineering

The first industrial success was the control of a cement kiln in Denmark, where operators' rules for burning zone temperature and oxygen were encoded as fuzzy rules [4]. The Sendai subway in Japan used predictive fuzzy control for automatic train operation, which improved riding comfort and stopping accuracy [5]. Fuzzy controllers later appeared in washing machines, cameras, heating systems, water treatment dosing and vehicle subsystems, and textbooks collect applications across civil, mechanical, chemical and electrical engineering [12].

Fuzzy control is most useful when a good mathematical model is missing but operator experience exists, when the process is nonlinear, and when smooth, explainable behaviour matters. It does not remove the need for analysis: Stability, robustness to disturbances and the effect of measurement noise must still be checked, for example by simulation over the operating range, as in the notebook. When a reliable model exists, classical or model-based control is often the better engineering choice.

Formulas: The tank under fuzzy control

The tank of the notebook and the lab has the cross-section $A = 2 \text{ m}^2$. The inflow is $q_{\text{in}} = q_{\text{max}} v$ with the valve opening v between 0 and 1 and $q_{\text{max}} = 1 \text{ m}^3/\text{s}$, and the outflow through the outlet follows Torricelli's law, $q_{\text{out}} = k \sqrt{h}$ with $k = 0.6$. The mass balance gives the rate of change of the level h :

$$A \frac{dh}{dt} = q_{\text{max}} v - k \sqrt{h}$$

The level is steady when inflow and outflow are equal, which requires the valve opening

$$v_{\text{ss}} = \frac{k \sqrt{h}}{q_{\text{max}}}$$

The fuzzy controller does not set the valve directly: It sets the change of the opening per second, Δv , so that in each time step of length Δt

$$v_{n+1} = v_n + \Delta v \times \Delta t, h_{n+1} = h_n + \frac{q_{\text{max}} v_{n+1} - k \sqrt{h_n}}{A} \Delta t$$

where the opening is kept between 0 and 1. The level equation is the Euler step of Week 1.

Worked example: The first control step after a change of the set point

The tank is at $h = 1 \text{ m}$ with $v = 0.6$, which is steady, because $0.6 \times \sqrt{1}/1 = 0.6$. The set point now jumps to 1.5 m.

Step 1. The target opening. To hold 1.5 m the valve must reach $v_{\text{ss}} = 0.6 \times \sqrt{1.5}/1 = 0.6 \times 1.224745 = 0.7348$.

Step 2. The inputs of the controller. The error is $e = 1.5 - 1 = 0.5 \text{ m}$, and the level does not move yet, so $r = 0$.

Step 3. Fuzzification and rules. For the error, $\mu_{\text{low}}(0.5) = 0.5$ and

$$\mu_{\text{okay}}(0.5) = \max \left(\min \left(\frac{0.5+0.4}{0.4}, \frac{0.4-0.5}{0.4} \right), 0 \right) = \max \left(\min(2.25, -0.25), 0 \right) = 0; \text{ for the rate,}$$

$$\mu_{\text{none}}(0) = 1. \text{ Only R1 fires, with } w_1 = 0.5; \text{ R3 gets } \min(0, 1) = 0.$$

Step 4. Defuzzification. The aggregated set is open fast clipped at 0.5, a trapezoid that is symmetric about the peak 0.05, so its centroid is $\Delta v = 0.05$ per second.

Step 5. One time step of $\Delta t = 0.5 \text{ s}$.

$$v_1 = 0.6 + 0.05 \times 0.5 = 0.625$$

$$h_1 = 1 + \frac{1 \times 0.625 - 0.6 \times \sqrt{1}}{2} \times 0.5 = 1 + \frac{0.625 - 0.6}{2} \times 0.5 = 1 + 0.00625 = 1.00625 \text{ m}$$

The level starts to rise. As it approaches the set point the error enters the set okay, rules R3 and R4 take over and the valve settles near 0.7348. Part D of the [interactive lab](#) simulates the whole response.

Python step 4: NumPy arrays and first plots

From lists to arrays

Engineering calculations usually apply one formula to many values. NumPy provides the array, a grid of numbers of one type on which arithmetic works element by element [8]. Multiplying an array by 10 multiplies every element, whereas multiplying a list by 10 would repeat the list ten times. Arrays have a **shape**, the number of elements along each axis, and a **dtype**, the type of their elements. `np.linspace(a, b, n)` creates `n` evenly spaced values from `a` to `b`, which is the usual way to sample a variable such as a tank level.

```
import numpy as np

levels_cm = np.array([12.0, 35.5, 50.0, 71.2, 94.0])
print(levels_cm * 10)           # millimetres, element by element
print(levels_cm.shape, levels_cm.dtype, levels_cm.mean())
x = np.linspace(0, 100, 11)     # 11 evenly spaced points from 0
to 100
print(x)
```

Output

```
[120.  355.  500.  712.  940.]
(5,) float64 52.54
[ 0.  10.  20.  30.  40.  50.  60.  70.  80.  90. 100.]
```

Functions that work on arrays

A function written with array operations accepts a single number as well as a whole array. The triangular membership function of fuzzy logic rises linearly from a foot `a` to a peak `b` and falls to a second foot `c`. `np.minimum` and `np.maximum` compare element by element, which expresses the whole shape in one line.

```
def tri(x, a, b, c):
    """Triangular membership function with feet a and c and peak b."""
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)),
0.0)

print(tri(35.0, 0, 25, 50))
print(np.round(tri(x, 0, 25, 50), 2))
```

Output

```
0.6
[0.  0.4 0.8 0.8 0.4 0.  0.  0.  0.  0.  0.]
```

Aggregation and boolean masks

Functions such as `np.sum`, `np.max` and `np.argmax` reduce an array to a single value or position. Comparisons produce boolean arrays, called masks, which count values when summed and select values

when used as an index. The centroid of a fuzzy set, the defuzzification method of the Mamdani controller, is a weighted mean: The sum of x times membership is divided by the sum of membership. The example clips the output set “open” of a valve at a rule strength of 0.6.

```
valve = np.linspace(0, 100, 201)          # valve opening in percent
open_set = tri(valve, 40, 70, 100)        # output set "open"
clipped = np.minimum(open_set, 0.6)       # a rule with strength 0.6
clips the set
centroid = np.sum(valve * clipped) / np.sum(clipped)
print(f"centroid: {centroid:.1f} % opening")
print("points above 0.5:", np.sum(clipped > 0.5))
print("openings with membership above 0.5:", valve[clipped > 0.5][[0, -1]])
```

Output

```
centroid: 70.0 % opening
points above 0.5: 59
openings with membership above 0.5: [55.5 84.5]
```

The last line uses the mask as an index to keep only the matching openings, and then selects the first and the last of them.

Plots

Matplotlib draws figures from arrays [9]. `plt.plot(x, y)` connects points with lines, the `label` argument names each curve for the legend, and axis labels with units belong on every engineering plot. The code below draws the three level sets of the tank controller; the notebook shows the figure.

```
import matplotlib.pyplot as plt

level = np.linspace(0, 100, 201)
plt.figure(figsize=(6, 2.8))
plt.plot(level, tri(level, -50, 0, 50), label="low")
plt.plot(level, tri(level, 0, 50, 100), label="medium")
plt.plot(level, tri(level, 50, 100, 150), label="high")
plt.xlabel("tank level (cm)")
plt.ylabel("membership")
plt.legend()
plt.show()
```

Feet placed outside the range, such as -50 and 150, make the outer sets start or end at full membership at the edges of the range, which is how a controller treats a tank that is empty or full.

Check your understanding. After `a = np.array([1.0, 2.0, 3.0])`, what is `a * 2 + 1`?

- A. `array([3., 5., 7.])`
- B. `[1.0, 2.0, 3.0, 1.0, 2.0, 3.0, 1]`
- C. An error
- D. 15.0

Answer: A. Arithmetic on arrays acts element by element. With a list, 2 would repeat the list, and adding 1 would raise an error.*

Discipline challenges

Fuzzy logic suits quantities that experts judge with words. Choose a row, define linguistic variables with membership functions, write a rule table, and test the result on a few cases.

Department	Challenge
Environmental Engineering	Coagulant dosing from raw-water turbidity and pH, or an air quality index from several pollutants.
Food Engineering	Drying control from product moisture and air temperature, or sensory quality grading from colour and texture scores.
Civil Engineering	Condition rating of a bridge element from crack width, corrosion and deflection.
Mining Engineering	Ground stability assessment from joint spacing, water inflow and blasting vibration.
Electrical and Electronics Engineering	Battery charging current from temperature and state of charge.
Mechanical Engineering and Automotive Engineering	Wheel slip control or engine cooling fan control from temperature and its rate.
Chemical Engineering	Reactor temperature control with a two-input fuzzy PI structure.
Industrial Engineering	Supplier selection with fuzzy scores for price, quality and delivery.
Textile Engineering	Fabric hand evaluation from bending, friction and compression measurements.
Earth Sciences Engineering and Geophysical Engineering	Landslide susceptibility from slope, lithology and rainfall classes.
Mathematics and Statistics	Compare t-norms and s-norms and prove which classical set laws fail for min and max.

Weekly task

Design a fuzzy controller or a fuzzy assessment model for a problem of your department with at least two inputs, three sets per input and a complete rule table. Implement it with the from-scratch code or with scikit-fuzzy, plot its control surface, and test it on a simple simulation or on at least ten cases. Report the design choices and the behaviour in about 500 words and cite at least three works from this week's references [3, 10, 12].

Research and report assignment (optional)

Fuzzy systems in engineering decision making. Review fuzzy control or fuzzy decision support in one domain, such as water treatment, building energy, traffic, mining safety or food quality. Compare Mamdani, Takagi-Sugeno and adaptive neuro-fuzzy approaches and discuss how the membership functions and rules were obtained, from experts, from data or by optimisation [6, 7, 11].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

[1] Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338-353. [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X)

- [2] Zadeh, L. A. (1973). Outline of a new approach to the analysis of complex systems and decision processes. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-3(1), 28-44. <https://doi.org/10.1109/TSMC.1973.5408575>
- [3] Mamdani, E. H., & Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1), 1-13. [https://doi.org/10.1016/S0020-7373\(75\)80002-2](https://doi.org/10.1016/S0020-7373(75)80002-2)
- [4] Holmblad, L. P., & Østergaard, J.-J. (1982). Control of a cement kiln by fuzzy logic. In *Fuzzy Information and Decision Processes* (M. M. Gupta & E. Sanchez, Eds.) (pp. 389-399). North-Holland.
- [5] Yasunobu, S., & Miyamoto, S. (1985). Automatic train operation system by predictive fuzzy control. In *Industrial Applications of Fuzzy Control* (M. Sugeno, Ed.) (pp. 1-18). North-Holland.
- [6] Takagi, T., & Sugeno, M. (1985). Fuzzy identification of systems and its applications to modeling and control. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-15(1), 116-132. <https://doi.org/10.1109/TSMC.1985.6313399>
- [7] Jang, J.-S. R. (1993). ANFIS: Adaptive-network-based fuzzy inference system. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(3), 665-685. <https://doi.org/10.1109/21.256541>
- [8] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- [9] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- [10] Lee, C. C. (1990). Fuzzy logic in control systems: Fuzzy logic controller, Part I. *IEEE Transactions on Systems, Man, and Cybernetics*, 20(2), 404-418. <https://doi.org/10.1109/21.52551>
- [11] Kose, U., & Arslan, A. (2017). Forecasting chaotic time series via ANFIS supported by vortex optimization algorithm: Applications on electroencephalogram time series. *Arabian Journal for Science and Engineering*, 42(8), 3103-3114. <https://doi.org/10.1007/s13369-016-2279-z>
- [12] Ross, T. J. (2017). *Fuzzy Logic with Engineering Applications* (4th ed.). Wiley.